FULL-STACK AI TRACK · DEEP DIVE DD8

Put the spec back in charge

Spec-Driven Development with AI

Participant Handbook · read before Wednesday

When	Wednesday 4 November 2026, 08:00 to 11:00 IST
Format	A hands-on **milestone lab**, mostly on your own: success criteria, hints and checkpoints, not a follow-along. No prompts are handed out. The facilitator checks in twice, after milestone 4 and after milestone 8.
Where	Virtual (Zoom or Teams), working on the Linux lab VM with VS Code and Claude Code
Before you come	About 50 to 65 minutes of reading, plus a 15-minute warm-up. **Nothing to install.** The lab kit is handed out at the start of the session.
Keep it open	Part 5 (the terminal handbook) and the one-page cheat sheet at the back sit next to your terminal on the day

PwC Emerging Business · upGrad Enterprise · Full-Stack AI Track · Delivery material

Contents

1. Start here
2. Part 1 · The session at a glance
3. Part 2 · Before you arrive
4. Part 3 · The ideas
5. Part 4 · How the lab works
6. Part 5 · The terminal handbook
7. Part 6 · The kit, mapped
8. Part 7 · Glossary
9. Part 8 · Answers, reading and the cheat sheet
10. One-page cheat sheet

If you only have 30 minutes: read **Part 1**, the five **"In plain English"** boxes in Part 3, and Part 5, sections 5.1 to 5.6.

Start here

What this handbook is for

On Wednesday 4 November you will spend three hours putting a spec back in charge of some code that has drifted away from it, and building the check that stops it drifting again. It is a hands-on session, and it is different from a follow-along: **nobody will give you prompts to paste, and nobody will stop the room after every step.** You will get goals, success criteria and a clock, and you will work out how to get there yourself, with Claude Code, by hand, or both.

That only works if you arrive knowing three things, and this handbook gives you all three:

1. **The ideas** (Part 3): what a spec is and how it differs from a PRD, what makes a requirement testable, and what traceability and drift mean.
2. **The method** (Part 4): how a milestone works, how the hints and the two check-ins work, and how to brief Claude Code well when the thing you are asking for is a document.
3. **The commands** (Part 5): every command you will type, what it does, why we run it, and how to read what it prints. A lot of them are git.

How long it takes. About 50 to 65 minutes to read properly, plus 15 minutes for the warm-up and the self-check in Part 2. If you have only 30 minutes, read Part 1 (the shape of the day), the five "In plain English" boxes in Part 3, and Part 5.1 to 5.6.

What you do not need to do. You do not need to install anything beforehand. The lab kit is handed out at the start of the day, and a minute later it is running. It has no dependencies, so there is not even an `npm ci`. This handbook needs nothing but a reader.

Keep it open on the day. Part 5 (the terminal handbook) and the one-page cheat sheet at the back are made to sit next to your terminal while you work.

What you will be able to do afterwards

By the end of the session you will be able to:

- find the places where a PRD, a team decision and shipped code disagree, and prove each one with a line of evidence;
- turn a PRD into a structured spec: stable requirement IDs, given/when/then criteria precise enough to test, explicit non-goals, and the vague sentences flagged instead of guessed;
- review an AI's reading of a requirement against what the product owner really meant, and record the review;
- write a task breakdown in which every task, and every test, cites the requirement it serves;
- build a traceability check that links each requirement to its code and its tests, and flags drift;
- prove a requirement's test was red before the fix, so you know it is a real check.

The last three are habits more than facts. They are the point of doing this as a lab.

Where this fits

This is DD8 of 33 sessions in the Full-Stack AI track. On **Monday** (DD7) you refactored a legacy function and learned that a green test run only tells you about what someone thought to test: today's lab is the other half of that lesson, because a spec is where "what to test" comes from. In **DD6** you built a grounded feature and wrote questions that could fail. In **DD2** you met Claude Code and context files, which you use all morning. Next comes **DD9** (Saturday 7 November, at **09:00**) on AI-assisted testing and debugging, then **DD10** (Wednesday 11 November) on evaluating AI features, which applies the same traceability discipline to AI behaviour. The evidence habit you practise today comes back in the **capstone**.

Part 1 · The session at a glance

The scenario

A retailer's "buy online, pick up in store" (BOPIS) feature has drifted. Three sprints ago a product owner wrote a PRD with eight rules about which items can be offered for pickup. Since then the team has agreed things in Slack that were never written back: a stock buffer was lowered, and a price cap was added after the fraud team asked. One rule from the PRD, about perishable items, was never built; and nobody remembers whether the weight limit was. A decision made in Slack and never written down is how three sprints of drift happen.

Your job over the morning is to find that drift, write the spec that the humans and the AI can both build from, close the gap with a test that names the requirement it checks, and build a script that finds the next gap before a customer does. Everything in the project is made up: the retailer, the people and the numbers.

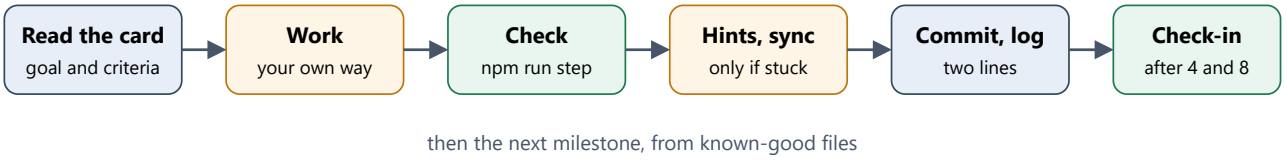
The agenda (Wednesday 4 November, 08:00 to 11:00 IST)

Time	What happens
08:00	**Kick-off.** The scenario, how the morning works, the ground rules
08:05	**Get the kit.** You receive a link, save a zip, and run four commands. It takes a minute while we carry on
08:09	**Concept sprint.** Why specs drift, a spec you can build from, traceability, humans in the loop, tests from criteria. A question first, then the answer
08:22	**Live demo.** A Slack decision that nobody wrote down, turned into a requirement, a rule and a test, worked as the method you will use
08:32	Break
08:39	**Lab briefing.** How the cards, the hints and the check-ins work
08:43	**Block A: milestones 1 to 4, on your own.** Find the drift, write the spec, review it with the product owner, plan the work
09:28	**Check-in 1.** The facilitator walks through the steps, the tests and the solutions for milestones 1 to 4
09:43	Break
09:49	**Block B: milestones 5 to 8, on your own.** Build the missing rule, test every criterion, build the traceability check, prove the test was red
10:28	**Check-in 2.** The same, for milestones 5 to 8
10:44	**Wrap-up.** Reading the output, tools, a short exit quiz, the homework, the close
11:00	End

About two thirds of the session is the lab, and most of that is you working by yourself. The talking is kept short because this handbook did the explaining.

How the lab works

Each of the eight milestones is written like a ticket in a real engineering team. It tells you the **goal**, why it matters, what you are **given**, the **interface contract** (the exact file names, headings and shapes that other documents and the checks depend on), and the **success criteria**. It does **not** tell you how to do it.



then the next milestone, from known-good files

Every milestone follows the same loop. The room only stops twice.

- **Read the card.** The success criteria are your acceptance criteria. Read the card before you touch the keyboard.
- **Work.** You decide how. Briefing Claude Code is a good way; so is doing it by hand. What matters is the result.
- **Check.** `npm run step -- 4` tells you which success criteria are not met yet. You never have to ask anyone whether you are done.
- **Hints and sync, only if you are stuck.** Three hints per milestone, which open a few minutes after you start it. You take them yourself with a command (`npm run hint -- 4`). And `npm run checkpoint -- 4` gives you the reference files, so the next milestone starts from a known-good place. Your own work is kept on a git branch first.
- **Commit and log.** A git commit for each document or change that is ready, and two or three lines in your own evidence log: what you tried, what you learned.
- **Check-in.** After milestone 4 and after milestone 8 the facilitator walks through the steps, the tests and the solutions, and shows the reference brief (the "correct prompt") next to what you wrote.

What is expected of you, and what you get

You are expected to	You get
Read each card, write your own briefs, read every output, and run the check	A written ticket for every milestone, and an automatic check that tells you which criteria are not met yet
Try before you take a hint, and say so when you take one early	Three hints per milestone, then the reference solution at the check-ins
Commit after each document or change that is ready	A history you can read (a spec's history is part of the lab), and a free sync
Say in the chat when you are stuck (it is not a failure to be stuck)	Teaching assistants watching the chat

How "done" is measured. A milestone is done when its check says **PASSED** (`npm run step -- 4` prints `STEP 4 PASSED`); `npm run progress` checks them all. Nobody grades your log. The session ends with a five-question exit quiz and a feedback form. A milestone reached by syncing still counts as understood **if you can explain it**.

Part 2 · Before you arrive

Readiness checklist

Tick these before Wednesday morning. None of them needs the lab kit.

- ☐ You can log in to the lab VM and open **VS Code** on it.
- ☐ The **Claude Code** extension is installed and you are **logged in**. Open the panel and say hello to check.
- ☐ In a terminal on the VM, `node --version` prints **v20 or higher**.
- ☐ In the same terminal, `git --version` prints a version. **This lab uses git all the time.**
- ☐ You know how to open a terminal in VS Code (Terminal menu, then New Terminal) and how to type `cd` and `ls` (Part 5.2).
- ☐ You have a folder where you can save a zip file, such as your home folder.
- ☐ You have read at least Part 1 and Part 5.1 to 5.6 of this handbook.
- ☐ You have done the warm-up below, or tried.

If any of the first four fail, tell the course coordinator **before** the session. They are environment problems, and we can fix them faster in advance than on the day.

Warm-up (about 15 minutes, nothing to install)

You only need a terminal with `node` and `git`. Three small exercises: a safety net, a sentence made precise, and a test that fails first.

Exercise 1 · git as a safety net, in ten commands

The lab leans on git, so get comfortable with five moves: save, look, put aside, bring back, read the history. Make a toy folder:

```
mkdir toy && cd toy
git init
printf '# Shipping\n\nOrders over 50 dollars ship free.\n' > spec.md
git add spec.md
git commit -m "First draft of the shipping spec"
```

If git says it does not know who you are, run `git config user.email "you@example.com"` and `git config user.name "Your Name"` and commit again.

Now change the file, look at the change, put it aside, and bring it back:

```
printf 'Express orders always pay.\n' >> spec.md
git diff
git stash
cat spec.md
git stash pop
git log --oneline
```

```
$ git diff
@@ -1,3 +1,4 @@
 # Shipping

 Orders over 50 dollars ship free.
+Express orders always pay.
$ git stash
Saved working directory and index state WIP on main: d17d0bc First draft of the shipping spec
$ cat spec.md
# Shipping

Orders over 50 dollars ship free.
$ git stash pop
no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (7bebcb6dfebe51015f041589ae0401009648faec)
```

(On Windows git may also print a line about LF and CRLF. It is harmless.) `git stash` puts your uncommitted changes aside and gives you the last commit back; `git stash pop` brings them back. You will use exactly this in milestone 8, to take a fix away for a minute.

Exercise 2 · From a sentence to a criterion (pencil and paper)

Here is one sentence from a made-up PRD: **"Orders over 50 dollars ship free."** Write two Given / When / Then criteria for it, each precise enough that someone could write a test from it without asking you a question. Look at the sentence carefully. Is "over" the same as "at least"? What happens at exactly 50? What if the order is 49.99? Compare yours with the answer in Part 8.

Exercise 3 · Red, then green

A test that was never red proves nothing. See what red looks like. Still in the `toy` folder:

```
echo '{"type":"module"}' > package.json
```

Create `shipping.js` with a function that does not work yet:

```
export function shippingCost(total) {
  return 4.99;
}
```

Create `shipping.test.js`. Notice that each test name starts with a requirement ID, followed by a colon:

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { shippingCost } from './shipping.js';

test('REQ-001: an order of 50.01 dollars ships free', () => {
  assert.equal(shippingCost(50.01), 0);
});

test('REQ-001: an order of exactly 50 dollars pays 4.99', () => {
  assert.equal(shippingCost(50), 4.99);
});
```

Run it:

```
node --test shipping.test.js
```

```
✖ REQ-001: an order of 50.01 dollars ships free (0.8695ms)
✓ REQ-001: an order of exactly 50 dollars pays 4.99 (0.1105ms)
ℹ tests 2
ℹ pass 1
ℹ fail 1
```

One test is **red**, because the function charges everyone. The other is green by luck: it passes even though the feature is missing, which tells you something about that test. Now make the function do what the sentence says, and run it again:

```
export function shippingCost(total) {
  return total > 50 ? 0 : 4.99;
}
```

```
✓ REQ-001: an order of 50.01 dollars ships free (0.505ms)
✓ REQ-001: an order of exactly 50 dollars pays 4.99 (0.0831ms)
ℹ tests 2
ℹ pass 2
ℹ fail 0
```

Finally, change `>` to `>=` and run it once more. Which test goes red, and why? That is what "a test that can fail" means.

Self-check (10 minutes)

Answer these without looking. The answers are in Part 8. If you get fewer than six, re-read the plain-English boxes in Part 3.

1. What is the difference between a PRD and a spec?
2. Why does a requirement have an ID, and why must it never be reused?
3. What is wrong with the requirement "the page should load quickly"? What would you do with that sentence?
4. A criterion says "Given a basket with enough items, when the customer checks out, then shipping is cheap". Name two things that stop you writing a test from it.
5. Why is "AI-suggested clarifications get reviewed by the product owner" a rule and not a nicety?
6. A code change alters how a discount is calculated, and no document changes. What do we call that, and what would catch it?
7. What does it mean if a test has never been red? Name one way to find out.
8. In `test('REQ-014: refunds are capped at the original price', ...)`, what does the `REQ-014:` do for you?

Part 3 · The ideas

The session leans on eight ideas. Each section ends with an **In plain English** box: if you remember nothing else, remember those. None of the lab's answers is in this chapter. The examples use a made-up shipping policy and a made-up loyalty scheme, not the BOPIS feature.

3.1 Which document is the truth?

Every team has at least three: what the PRD says, what people agreed in a meeting or on Slack, and what the code actually does. At the start of a project they agree. After three sprints they do not, and nobody noticed when they stopped. A decision made in a chat thread and never written back did not just get lost: it became a bug with a head start.

The cure is not a better PRD. It is **one document that the humans and the AI both build from**, that changes only when someone who owns it says so, and that you can check the code against. That document is a *spec*.

> **In plain English.** If three places can disagree, they will. Pick one to be the truth, make it checkable, and make changing it a visible event.

3.2 A PRD is not a spec

	A PRD	A spec
Written for	People deciding what to build	People and tools building and testing it
Style	Prose, goals, background	One requirement at a time, each with an ID and criteria
Precision	"Should be fast", "most large items"	"Over 20 kg", "within 200 ms at the 95th percentile"
Changes	Rewritten, sometimes quietly	Versioned: a change is visible, with a reason
Answers	"What are we trying to do?"	"What exactly must be true, and how will we check?"

An AI can turn a PRD into code. It will do it fluently, and it will fill every gap with a guess that looks reasonable. **Vague requirements just produce vague code faster.** That is why there is an intermediate step: PRD to spec, with a human in the loop, *then* spec to code.

> **In plain English.** Prose is for deciding. A spec is for building and checking. Put the spec between the PRD and the code.

3.3 A requirement you can test

A good requirement in a spec has four parts:

1. **An ID** that never changes: `REQ-014`. It names this requirement forever, even if its words change, and it is never reused. Everything downstream (tasks, code comments, test names) carries it.

2. **A one-line statement** of what must hold.
3. **Acceptance criteria in Given / When / Then form**, one for each case you would want to test. *Given* a situation, *when* something happens, *then* a result you could observe.
4. **Numbers wherever there is a boundary.** "A basket over 50 dollars" is a thing you can test; "a big basket" is not.

```
## REQ-014: Refunds are capped at the original price

A refund may not be more than the price the customer paid.

- **Given** an order item paid at 40.00
- **When** a refund of 40.00 is requested
- **Then** the refund is approved

- **Given** an order item paid at 40.00
- **When** a refund of 40.01 is requested
- **Then** the refund is refused
```

Notice how the two criteria sit on either side of the boundary. Each one becomes exactly one test. **Given / When / Then is not bureaucracy; it is the difference between a requirement you can generate a real test from and one you can only argue about.**

> **In plain English.** An ID so you can point at it. A line so you know what it is. Criteria with numbers so you can test it.

3.4 What to do with a sentence you cannot test

Some sentences in a PRD cannot become requirements as they stand: "the checkout should feel smooth", "most orders should ship quickly". There are only three honest things to do with one:

- **Flag it.** List it in the spec under "too vague to test", with the question it raises. Do not guess.
- **Make it testable** once someone with the authority has answered the question ("quickly means within 2 days").
- **Drop it** as out of scope, if the owner says it is not part of this feature.

The tempting wrong thing is to let the AI choose a meaning and write it as a requirement. A requirement too vague to test is one you are guessing at, not one you are building. Keep the **non-goals** next to the goals for the same reason: they stop an AI building more than you asked for.

> **In plain English.** If you cannot say how you would test it, you do not have a requirement. Flag it; do not guess.

3.5 Humans in the loop: who owns the spec

A spec needs a **named owner** who approves every change to it. When an AI reads an ambiguous sentence and suggests what it means, that suggestion is useful, but until the owner has looked at it, it is a *guess wearing the spec's authority*. The spec review, not the code review, is where intent is locked in. By the time the code is reviewed, the wrong meaning is already built.

Two habits make this real. A **changelog** inside the spec, with a line for each change and the reason for it, so that nothing is changed silently. And **git**: commit the first draft the AI produced, then commit the reviewed version. The difference between the two commits is exactly what the review found.

> **In plain English.** The AI suggests; the owner decides; git remembers. A change to a requirement should always leave a trace.

3.6 From spec to tasks to code

Once the spec is settled, a **task breakdown** maps each requirement to the work: which files it touches, what tests it needs, and which requirements are already built and only need a test. Two rules keep this honest. **Every task cites a requirement**, because work with no requirement is work nobody can explain later. And **do not plan code for what already exists**: a plan that rebuilds working code is how code drifts from its spec in the first place.

The IDs then travel. A comment `// REQ-014` next to the rule that implements it. A test whose name starts `REQ-014: ...`. Now "why does this code exist?" has a one-line answer, and "what tests this requirement?" has one too. **Code with no requirement ID attached is code nobody can explain in six months, including the person who wrote it.**

> **In plain English.** Carry the ID everywhere: the task, the code comment, the test name. Then every question has a lookup.

3.7 Tests from criteria, and red first

Because each criterion is precise, the test can be written **before** the code, straight from the criterion, with its assertions worded close to the criterion's own language. Then you run it and it must be **red**. That is the proof the test is checking the requirement: a test that passes before the code exists is checking nothing.

```
✕ REQ-001: an order of 50.01 dollars ships free
✓ REQ-001: an order of exactly 50 dollars pays 4.99
```

Look at the second one: it is green while the feature is missing, which is correct, because it describes what must *not* change. So red-first is not "all tests red"; it is "the tests about the new behaviour are red". A test that was never red, and could not be made red, is a test worth being suspicious of.

A second check goes further. Take a rule and make it slightly wrong (a `>` where the spec says `>=`, a 5 where it says 6). If no test goes red, the tests were not really about that rule, whatever their names say. The lab does this to your tests.

> **In plain English.** Write the test first, watch it fail, then make it pass. A test that has never failed has never proved anything.

3.8 Traceability: a check, not a table

A traceability table links each requirement to the code that implements it and the tests that verify it, and flags any requirement with no code or no test. The important word is *check*. A table made once at kickoff is already lying by sprint two, because nobody updates it. So make it a **script that runs on every change**, reads the files each time, and fails when something is not linked.

A good traceability check also notices the mirror image: an ID in the code or the tests that **no requirement owns** (an *orphan*). And it can watch for **drift** itself: if the code under `src/` changed between two commits and nothing under `spec/` did, either the change did not touch behaviour, or a decision was just made without being written down. Run in a pull request, that question gets asked before the merge instead of three sprints after.

> **In plain English.** Make the table a script. It should fail when a requirement has no code, no test, or an owner-less ID, and it should ask what happened when the code moves and the spec does not.

3.9 The same ideas at work

On a real project, the same pieces exist under other names. Requirements live in a work-item tracker with stable IDs (Azure Boards or a similar tool). The spec is a Markdown file in the repository, reviewed in a pull request by its owner. The traceability and drift checks run in the build pipeline on every pull request. An AI assistant helps at every step, and a person approves at two: the spec, and the merge.

Ten questions to ask of any AI-assisted change are in Part 8. The short version: *Which requirement is this? Who agreed to it? What test would fail if it were wrong? And would anything notice if the code moved and the spec did not?*

Part 4 · How the lab works

4.1 Anatomy of a milestone card

Every milestone is a card in `LAB-GUIDE.md`, written like a ticket. Here is one for a made-up task that is **not** part of the lab, so you can see what each section is for.

Milestone X · Write down the shipping rules *(an example, not a real milestone)*

Ticket SHOP-000 · aim for **8 minutes** · hints open at **+3, +5, +7** *The pace to aim for, and when each hint opens (counted from when you start the milestone).*

Why this matters. The shipping rules live in three people's heads, so every change is a surprise. *The reason, so you can judge trade-offs.*

Your goal. A short document that lists each shipping rule as a requirement you could write a test from. *One outcome, in one sentence.*

You are given. The old PRD, a Slack export, and Claude Code. *Everything you may use, so you never have to guess what exists.*

Interface contract. The file is `docs/shipping-rules.md`. Each rule is a heading `## SHIP-001: Title`, then one line, then a Given / When / Then. Commit it. *The exact names, files and headings that other documents and the checks depend on. These are requirements, and they are not negotiable.*

Success criteria.

- **Must:** ☐ every rule has an ID and a Given / When / Then ☐ no rule is "should be quick" ☐ it is committed ☐ `npm run step -- X` passes
- **Stretch:** ☐ list the sentences you could not turn into a rule, with a question for each *How you will know you are done. "Must" is the finish line. "Stretch" is for when you are early.*

Where to start. Which sentences could you write a test for tomorrow? Which could not? *Questions to settle before you type. They are not instructions.*

Verify yourself. `npm run step -- X` *The command that tells you which criteria are not met.*

The things a card does **not** contain: a prompt to paste, a solution, or the names of the traps. Finding those is the milestone.

4.2 Working mostly on your own

This lab is built so that you can work at your own pace, without anyone stopping the room after every step.

Block	Milestones	About how long	Then
A	1, 2, 3, 4	45 minutes	**Check-in 1:** the facilitator walks through the steps, the tests and the solutions for 1 to 4
B	5, 6, 7, 8	39 minutes	**Check-in 2:** the same, for 5 to 8

- **The pace on each card is a guide, not a deadline.** Some milestones will take you less, some more. A slow milestone costs you nothing, because syncing is free and the next milestone starts from known-good files.
- **You do not need permission to move on.** When `npm run step -- N` says PASSED, go on to the next milestone.
- **Pace marks in the chat.** At a few set times the facilitator posts a line such as "by now, aim to be starting milestone 3". It is there so that you can see how you are doing against the room. You do not need to reply.
- **At the check-ins** the facilitator shows the reference solutions and the reference briefs, and compares them with what you built. They may ask you to paste your `npm run progress` line in the chat (see 4.8).
- **If you finish a block early,** do the stretch goals on the cards. They are the best use of spare time. Or help a neighbour.

4.3 The hint ladder

Each milestone has **three hints** and then a **reveal**. They are meant to be taken in order, each only when you need it.

Level	What it gives you	Example (for the made-up milestone above)
Hint 1: Nudge	Where to look, or the right question to ask	"Which of these sentences could you write a test for tomorrow?"
Hint 2: Approach	The shape of the solution, without the answer	"Ask the AI to flag any sentence it cannot test, instead of guessing."
Hint 3: Near-solution	Almost the answer, enough to finish	"A rule with a number in it, like 'over 50 dollars', is testable. 'Quick' is not."
Reveal	The reference brief (a good prompt), the approach, and the common mistakes	Shown by the facilitator at the check-ins

How the clock works: it starts for a milestone the first time you run `npm run start -- 4`, `npm run step -- 4` or `npm run hint -- 4`. Each hint opens a few minutes after that. If you ask too early, the command tells you how long to wait:

```
Milestone 1 · Find the drift
Hint 1 of 3 opens in about 5 minutes (the clock for this milestone started 0 minutes ago).
Try the questions under "Where to start" on the card first. That is where most of the learning is.
If you really need it now:  npm run hint -- 1 --now   (it is noted as an early hint, and you can
say so in LAB-LOG.md)
```

That is a nudge, not a lock. `--now` always works, and the early hint is counted in your `npm run progress` table.

How to use hints well:

- **Try first.** A few minutes of being stuck is where most of the learning happens.
- **Take them with the command:** `npm run hint -- 4` gives the next one for milestone 4. `npm run hint -- 4 --again` re-reads the ones you have taken.

- **If you take one early, write it in your log.** That is not a penalty: it is how you will remember what you needed.
- **The reveal** is `npm run hint -- 4 --reveal`. It is meant for the check-ins, when the facilitator shows it. Reading it early is allowed, and honest logging is all that is asked.

4.4 Syncing: catching up for free

If you are not where you want to be, you do not fall behind:

```
npm run checkpoint -- 4
```

This replaces your work with the reference solution for milestone 4, so milestone 5 starts from known-good files. Because this lab is built on git, it does this **with real commits**: it goes back to the starting commit and replays the reference solution one step at a time, so the history looks like the work of a careful engineer, with one commit for each document or change. That matters, because the spec's history is part of what the checks look at.

Your own work is kept. Everything you had committed, and anything you had not committed yet, goes onto a branch called `my-work-<time>`. `git branch` lists it, and `git log my-work-<time> --oneline` shows what was on it.

Using a checkpoint is not cheating. It is the reference solution, and the next milestone is built on it. What matters is that you understand it before you move on: read it, and write one line in your log about what it does differently from your version. You can sync at any time, not only at the check-ins.

4.5 Working with Claude Code on a spec

Claude Code is a very fast, very confident colleague. It reads the folder you open, follows the rules in `CLAUDE.md`, runs commands, and edits files. It is also sometimes wrong, and it will not always tell you. When you ask it to write a **document** (a spec, a plan), its mistakes look like good writing. Your job is not to type; it is to **decide what is true, and check that the document says it**.

Writing a brief

Because there is no prompt to paste, the card's success criteria become the acceptance criteria in **your** brief. A good brief has five parts:

Part	What to say	Example
Context	What exists, and what it is for	"This is a Node project. The PRD is `docs/prd.md`. My drift report is `docs/drift-report.md`."
Task	One thing to do, in one sentence	"Convert the PRD into `spec/bopis-eligibility.md`."
Constraints	What it must not do	"Flag any sentence you cannot test; do not guess what it means. Do not invent requirements. Do not edit the PRD."
Acceptance	The observable results that mean "done": headings, IDs, what a run prints	"One `## REQ-001` heading for each rule, a one-line statement, then Given / When / Then with numbers."

Part	What to say	Example
Verify	The commands or checks that prove it	"Then I will run `npm run step -- 2`."

Five habits that matter more than any wording

1. **Ask for a list of what it is unsure of.** An AI that has to name its guesses makes fewer of them silently.
2. **One document, or one change, at a time.** A small output can really be read.
3. **Make it quote.** "For each claim, quote the line of code that shows it." Then check three quotes yourself.
4. **Commit the first draft before you edit it.** The history then shows what the AI wrote and what a person changed.
5. **Read every output as if a colleague wrote it and went home.** Ask of each line: which requirement is this, and who agreed to it?

When Claude is confidently wrong

It will sometimes say a requirement is built when it is not (because a helper with the right name exists), or turn a vague sentence into a confident requirement, or give a rule a neat definition that nobody agreed. None of that is malice; it is what a fluent writer does with an ambiguous source. The defences are the ones in Part 3: criteria with numbers, a flagged list of what is unclear, a human who owns the spec, and tests that are able to fail.

Two safety rules

1. **Read the diff before you commit.** In this lab that means every file Claude touched, not only the one you asked about.
2. **Never paste a real key, password or customer's data into Claude or the chat.** Everything in the kit is made up, so you do not need to.

4.6 Reading a failing check

When `npm run step -- N` fails it prints which success criteria are not met yet. Here is a real one, from milestone 1, run before any report existed:

```
MILESTONE 1: Find the drift

✕ docs/drift-report.md exists and has a table with an item, a status and the evidence for each row
(3.7ms)
✕ every requirement of the PRD has a row, and so does the rule that only the code knows about
(5.7ms)
...
✕ failing tests:

✕ docs/drift-report.md exists and has a table with an item, a status and the evidence for each row
(3.7ms)
  AssertionError [ERR_ASSERTION]: docs/drift-report.md is missing. Create it with the table from
the milestone card.

STEP 1 not passing yet. Read the first failure above: it says which success criterion is not met.
Stuck? npm run hint -- 1 gives you the next hint when it opens. To catch up, npm run checkpoint --
1 gives you the reference.
```

- **The ✓ and ✕ lines** are the individual criteria. Fix them from the top.
- `pass` **and** `fail` are counts. You want `fail 0`.
- **The message after** `AssertionError` is the useful sentence. It says what is wrong, and for a document it says what the check was looking for. It never prints the answer: for milestone 1 it will not tell you which status is the right one.
- **For code, the check runs your function** on thousands of sample items and compares it with a hidden answer key. If it differs, it shows you an item and what it should have said.
- **The last two lines** tell you what to do next.

A failing check is information, not a verdict. Most are fixed by reading the message slowly.

4.7 The evidence log

`LAB-LOG.md` is your own record. After each milestone, write two or three lines: how you did it (your brief, or by hand), whether you took a hint (and whether it was early), and what you learned or what surprised you. Anything that looks wrong but is not part of the milestone goes under "Noticed". Nobody marks it. It is how you remember the lesson on Friday, and it is the habit the capstone's delivery narrative will ask for.

4.8 Asking for help, and the pulse

- **Say what you are stuck on.** "Stuck on 3: the check says the perishable requirement still decides by date" gets a better answer than "it's broken".
- **Teaching assistants** watch the chat and answer in one line. They will not give you an answer, but they will ask the question that gets you to it. They will also help with anything about your machine.
- **The pulse.** Shortly before each check-in you may be asked to run `npm run progress` and paste the **last line** in the chat. It looks like this:

```
PULSE M1[x] M2[x] M3[x] M4[ ] M5[ ] M6[ ] M7[ ] M8[ ] hints=2
```

It tells the facilitator where the room is, and nobody else can see your name against a result. It takes a few seconds to a minute to run, because it runs the check for every milestone.

Part 5 · The terminal handbook

Every command you will type on Wednesday, what it does, why we run it, and how to read what it prints. The lab VM runs Linux, and every command here is written for its shell (bash). The examples show real output from the lab kit, so what you see on the day will look the same, apart from things like dates, commit ids and file paths.

Each entry has the same shape:

- **What it does**, in plain English
- **Why we run it** in this lab
- **How to read the output**
- **If it goes wrong**

5.1 How to read a command

A command is a program, then what to do, then the details:

```
npm run step -- 4
|   |   |    |  └─ an argument: the number 4
|   |   |    └──── "--": everything after this goes to the script, not to npm
|   |   └───────── the name of a script defined in package.json
|   └───────────── a subcommand: run a script
└───────────────── the program
```

Piece	Looks like	Meaning
Program	`npm`, `node`, `git`	The thing being run
Subcommand	`run`, `commit`, `stash`	What to ask the program to do
Flag	`--grep`, `--all`, `--now`	Changes how it behaves. Some take a value
Argument	`4`, `src/bopis/eligibility.js`	What to act on
Comment	`# like this`	Ignored by the shell: it is there for you

Four things that catch people out:

1. **Quotes matter.** Inside **single quotes** (`'...'`) everything is literal. Inside **double quotes** (`"..."`) the shell replaces `$VARIABLES` with their values. A commit message with spaces needs quotes: `git commit -m "Add the drift report"`.
2. **A backslash at the end of a line** (`\`) means "this command carries on on the next line". It must be the last character on the line, with no space after it.
3. **Copy-paste can change characters.** Word processors and chat tools turn straight quotes into curly ones, and two dashes into one long dash. Either breaks a command. If a pasted command fails mysteriously, retype the quotes and dashes.
4. **Case and spaces matter.** `npm run step -- 4` is not `npm run step --4`.

> **Tip: tab completion.** Press **Tab** while typing a file or folder name and the shell finishes it for you. Press it twice to see the options.

5.2 Moving around and looking at files

Command	What it does	Why we run it
`pwd`	Prints the folder you are in	To check you are in `bopis-service`, not one level up (the most common mistake)
`ls -A`	Lists the files in the folder, including hidden ones (names starting with a dot)	To see what is there. `-A` shows hidden files such as `.gitignore`
`cd dd8-lab/bopis-service`	Moves into a folder. `cd ..` moves up one level, `cd ~` goes home	To get to the right place before every npm command
`cat docs/slack-thread.md`	Prints a file's contents	To read a small file without opening an editor
`head -n 20 file`	Prints the first 20 lines of a file or of a command's output	To look at the top of something long
`grep "supplier" src/bopis/eligibility.js`	Prints the lines of a file that contain some text	To search from the terminal (a word search finds a word, not a rule that is enforced)
Ctrl+C	Stops the program that is running in this terminal	To get out of anything that is stuck
`clear`	Clears the screen	To tidy up

If it goes wrong: `No such file or directory` almost always means you are in the wrong folder. Run `pwd`, then `ls`, then `cd` to where you meant to be.

5.3 Getting the kit

You will be given a link and a block of commands on the day. Here is what each one does.

```
cd ~
```

Go to your home folder, so the kit goes somewhere predictable.

```
unzip -q dd8-lab-kit.zip
```

Unpack the zip. `-q` means quiet: it does not list every file. If `unzip` is not installed, use `python3 -m zipfile -e dd8-lab-kit.zip .`, which does the same job.

```
cd dd8-lab/bopis-service
```

Move into the folder you will work in. **Only this folder is opened in VS Code.** The folders next to it (`hints`, `checks`, `checkpoints`) hold answers, including the product owner's, and Claude Code reads everything in the folder you open.

```
$ ls
README.md
bopis-service
checkpoints
checks
hints

$ cd bopis-service && ls -A
.gitattributes
.gitignore
.npmrc
CLAUDE.md
LAB-GUIDE.md
LAB-LOG.md
docs
package.json
scripts
spec
src
tests
tools
```

There is **no `npm ci` step** in this lab: the project has no dependencies, so there is nothing to download. It is Node's own test runner and a few plain scripts.

5.4 Node and npm

Command	What it does	Why we run it
`node --version`	Prints the Node version, for example `v24.14.1`	To check Node is new enough (v20 or higher)
`npm test`	Runs every test in the project	To confirm the starting point is healthy (`pass 13`), and to protect you from breaking things
`npm test -- --grep REQ-003`	Runs only the tests whose name contains `REQ-003`	To run the tests of one requirement
`npm run <name>`	Runs a script named in `package.json`	Almost everything in the lab is a script: `step`, `hint`, `po`, `trace` ...
`npm run help`	Prints every lab command with a one-line explanation	When you forget one

`npm test`

```
✓ kit: the PRD has eight numbered requirements (1.0782ms)
✓ kit: the catalog knows eleven categories (0.1135ms)
✓ an ordinary item is eligible (1.4009ms)
✓ a gift card is not eligible (0.0666ms)
...
ℹ tests 13
ℹ pass 13
ℹ fail 0
```

How to read it: each `✓` is a test that passed and each `✕` one that failed. The last block is the summary: you want `fail 0`. There are three families of tests: `tests/baseline` are the lab's own sanity tests, `tests/existing` are the team's tests of the shipped code, and `tests/spec` are yours (there are none until milestone 5). They stay green all session.

The double dash

```
npm run step -- 4
```

Without `--`, npm would try to interpret `4` itself. The `--` means "everything after this belongs to the script". Whenever you pass an option or argument to an npm script, put `--` before it. (`npm run step -- 4`, `npm test -- --grep REQ-003`, `npm run hint -- 2 --now`.)

Running only some tests: `--grep`

```
npm test -- --grep REQ-003
```

```
✓ .../tests/baseline/kit.test.js
✓ .../tests/existing/eligibility.test.js
✓ REQ-003: produce, dairy, meat, frozen and bakery items are not eligible (0.7682ms)
✓ REQ-003: a frozen item is not eligible, even though its shelf life is 180 days (0.1111ms)
✓ REQ-003: a pantry item is eligible, even when its date is 3 days away (0.0843ms)
✓ REQ-003: an item in a category that is not on the perishable list is eligible (0.6481ms)
ℹ tests 6
ℹ pass 6
ℹ fail 0
```

How to read it: `--grep` keeps only the tests whose **name** contains the text. That is exactly why the lab asks you to start every test name with its requirement ID: `--grep REQ-003` is then "run the tests of requirement 3". The two lines that show a file path are files with no matching test: node lists them and counts them, so `tests 6` here is four real tests and two file lines. (Once you have tests of your own in `tests/spec`, there is a third file line.) What you want to see is `fail 0`. `npm test -- --grep REQ-` runs every requirement-linked test.

5.5 git, the commands you will use

This lab uses git in every milestone, because a spec's history is part of the evidence. You need nine commands.

Command	What it does	Why we run it
`git status --short`	Lists the files that changed since the last commit	To see what you have changed and not yet committed
`git add -A`	Chooses every change for the next commit	Before a commit
`git commit -m "message"`	Saves the chosen changes as a commit	After each document or change that is ready
`git log --oneline`	The history, newest first, one line per commit	To see your steps, and to find the starting commit at the bottom
`git diff`	Shows exactly what changed since the last commit	To review a change before you commit it
`git show <commit>:<file>`	Prints a file as it was at an earlier commit	To read the shipped code after you have changed it
`git stash` / `git stash pop`	Puts uncommitted changes aside / brings them back	Milestone 8: to take the fix away for a minute
`git checkout <commit> -- <file>`	Puts one file back as it was at a commit	Milestone 8, if the fix is already committed
`git branch`	Lists branches	To find the `my-work-...` branch a sync made for you

`git status --short`, `git add`, `git commit`

```
$ git status --short
?? docs/drift-report.md
$ git add -A
$ git commit -m "Add the drift report"
```

How to read it: `??` means a new file git does not know yet; `M` means a file you changed. After `git add -A` and a commit, `git status --short` prints nothing. Commit after each document or change that is ready, with a message that says what it is. The lab's checks look at your commits, and this lab needs **two commits of the spec** (the AI's draft, then your review of it).

`git log --oneline`

```
$ git log --oneline
cc14421 Review the spec against the product owner's answers
29e0edf Convert the PRD into a structured spec (first draft)
546641c Record the drift between the PRD, Slack and the code
9f1596d Shipped code as delivered
```

How to read it: the newest commit is on top, and the **starting commit** (`Shipped code as delivered`, made by `npm run setup`) is at the bottom. Your ids will differ. Reading the history of a spec is how you see how a requirement changed.

`git diff`

```
git diff HEAD~1 --stat
```

```
 docs/spec-review.md       | 16 +++++++++++++++
 spec/bopis-eligibility.md | 56 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++-------------
 2 files changed, 57 insertions(+), 15 deletions(-)
```

How to read it: `git diff` alone shows what you changed since the last commit, line by line (`+` added, `-` removed). With `HEAD~1` it compares with the commit before the latest one, and `--stat` summarises by file. **Read the diff before you commit.** For a spec, this is the review: the diff between the AI's draft and yours is exactly what you changed.

`git stash`, and putting one file back

You will use this in milestone 8, to take a fix away and bring it back.

```
$ git status --short
 M src/bopis/catalog.js
$ git stash
Saved working directory and index state WIP on main: 0c80ed3 Implement REQ-003: perishable items
are not eligible
$ git status --short
$ git stash list
stash@{0}: WIP on main: 0c80ed3 Implement REQ-003: perishable items are not eligible
$ git stash pop
Dropped refs/stash@{0} (0701806bff47abe9ff257a7a7b66e9cfcbc8a80a)
```

How to read it: `git stash` hides your uncommitted changes (the status is now empty), and `git stash pop` brings them back. **If you forget the pop, your work is hidden, not lost:** `git stash list` shows it. If the fix is already committed there is nothing to stash; put the old file back instead, and then put the new one back:

```
git checkout 9f1596d -- src/bopis/eligibility.js    # the old version, from the starting commit
git checkout HEAD -- src/bopis/eligibility.js       # the current version again
```

If it goes wrong: `git commit` saying "Please tell me who you are" means git has no name or email. `npm run setup` sets one for this folder; if it did not, run `git config user.email "you@example.com"` and `git config user.name "Your Name"` inside the project.

`git reset --soft`, when a mistake is already in the history

If you committed something wrong and a later commit fixed it, the first commit is still in the history. `git reset --soft <commit>` moves the branch back to that commit and **keeps all your files as they are**, so you can commit them again as one clean commit. Use it only in the lab repository, which has no remote: rewriting history other people have already pulled makes trouble for them.

5.6 The lab commands

`npm run setup`

What it does: makes the starting git commit, "Shipped code as delivered". **Why we run it:** so that every change you make shows up in `git status` and `git diff`, and so the checks have a baseline to compare with. It refuses if the code or the documents were changed first, and sets a name and email for this folder if git had none.

```
git had no name or email on this machine, so this repository now uses "Lab learner
<learner@lab.local>" for your commits.
Done. The starting commit is 9f1596d498: "Shipped code as delivered".
Everything you change from now on shows up in git status and git diff. Commit after each document
or change that is ready.
```

(The first line appears only if git had no name and email.) Run it **once, before you change anything**.

`npm run doctor`

What it does: checks Node, git and the kit layout. **Why we run it:** it is the quickest way to find out why nothing works. It ends with `Ready.` when everything is fine.

```
  ok    Node 24.14.1
  ok    the shipped code, src/bopis/eligibility.js, is there
  ok    the PRD, docs/prd.md, is there
  ok    checks/, checkpoints/ and hints/ found next to this folder
  ok    git version 2.53.0
  ok    the starting commit exists (9f1596d498)

Ready. Open LAB-GUIDE.md and start milestone 1.
```

`npm run po`

What it does: prints the product owner's answers to the team's questions. **Why we run it:** in milestone 3, to compare your spec with what the owner actually meant. The answers live next to the folder you open, so that Claude Code does not read them while you are still working from the PRD alone. Run it when a milestone card tells you to, and paste what you need into your brief.

```
# Product owner's answers

*Dana Whitfield, product owner. Written after the team asked what the PRD really meant. These
answers are the product owner's
actual intent. Where they differ from the PRD, they win.*
...
```

`npm run start -- N`

What it does: starts the clock for milestone N and says when its hints open. **Why we run it:** it is optional. The first `step` or `hint` command for that milestone starts the clock for you.

```
Milestone 3 · Review against real intent
The clock for this milestone just started.
Aim to finish in about 10 minutes. Hints open after 5, 7, 9 minutes (a nudge, not a lock: npm run
hint -- 3).
Open LAB-GUIDE.md at the card for this milestone, and read it before you type anything.
```

`npm run step -- N`

What it does: runs the hidden check for milestone N and says which success criteria are not met. **Why we run it:** it is how you know you are done, without asking anyone. Part 4.6 explains how to read a failure.

```
MILESTONE 2: Convert the PRD to a spec

✓ the spec exists and every requirement has an ID of the form REQ-001, and no two share one
(3.0533ms)
✓ each requirement is one clear statement followed by at least one given/when/then criterion
(1.1068ms)
✓ the spec covers what the PRD asks for, one requirement for each thing (0.9348ms)
✓ the vague sentences are flagged for the product owner, and are not turned into requirements
(0.8711ms)
✓ the non-goals are written down, so the AI does not build more than was asked (0.7879ms)
✓ the spec has a named owner and a changelog, and it is committed (38.3442ms)
ℹ tests 6
ℹ pass 6

STEP 2 PASSED
Milestone 2 is done. Write two lines in LAB-LOG.md, commit, then go on to the next milestone.
```

Milestones 1 to 4 check your documents against facts the kit holds. Milestones 5 to 9 also **run** your code and your tests. A check takes a few seconds; the one for milestone 6 takes longer because it tries your tests against sixteen slightly wrong versions of the rules.

`npm run progress`

What it does: runs the check for every milestone and prints where you are, plus one line to paste in the chat. **Why we run it:** the facilitator asks for the last line shortly before each check-in.

```
  [x]  1  Find the drift                    done  (6 pass, 0 fail)  hints 0
  [x]  2  Convert the PRD to a spec         done  (6 pass, 0 fail)  hints 0
  [ ]  3  Review against real intent        not yet  (4 pass, 1 fail)  hints 0
  ...

2 of 8 milestones done, 0 hints taken.

PASTE THIS IN THE CHAT:
PULSE M1[x] M2[x] M3[ ] M4[ ] M5[ ] M6[ ] M7[ ] M8[ ] hints=0
```

It reads your work and never changes it. It takes a few seconds on a fast machine and up to a minute on a slow one.

`npm run hint -- N`

What it does: gives you the next hint for milestone N. **Why we run it:** when you are stuck, after you have tried. Part 4.3 explains the ladder.

```
Milestone 1 · Find the drift
Hint 1 of 3 · Nudge  (early)
--------------------------------------------------------------------------------
Make three lists before you write anything: what the PRD asks for (sections 1
to 8), what the code does, and what the Slack thread decided. Then compare
them two at a time. ...
--------------------------------------------------------------------------------
```

Flags: `--now` takes a hint before it has opened (it is noted as early); `--again` re-reads the hints you have taken; `--reveal` shows the reference brief.

`npm run checkpoint -- N`

What it does: replaces your work with the reference solution for milestone N, as real commits, keeping your own work on a branch. **Why we run it:** to catch up for free (Part 4.4).

```
Your code now matches the reference solution for milestone 2, in 2 commits on top of the starting
commit.
Your own work was kept on the branch my-work-2026-11-03T04-09-58  (git log my-work-2026-11-03T04-
09-58 --oneline shows it).
Run `npm test` to confirm, then carry on with the next milestone.
```

`npm run trace`

What it does: runs **your own** traceability check, `tools/trace.mjs`, which you build in milestone 7. **Why we run it:** it is the check that finds drift. Until you have built it, it only says so. This is what a good one prints, from the reference solution, one milestone before the weight limit is built:

```
Requirement Code                        Tests  Status
REQ-001     src/bopis/eligibility.js    3      OK
REQ-002     src/bopis/eligibility.js    2      OK
REQ-003     src/bopis/eligibility.js    4      OK
...
REQ-008     -                           0      NO CODE, NO TEST
REQ-009     src/bopis/eligibility.js    3      OK

1 requirement(s) not fully traced.
```

How to read it: one row for each requirement in the spec. `Code` is where its ID is written in `src/`, `Tests` is how many tests are named after it, and `Status` is `OK` only if both exist. The exit code is 1 when anything is not traced, which is how a build would fail. The card tells you the exact options (`--json`, `--root`, `--base`) it must support.

`npm run help`

Prints every lab command with a one-line explanation. It is the same list as the cheat sheet at the back of this handbook.

5.7 When something goes wrong

What you see	Most likely cause	What to do
`No such file or directory`	You are in the wrong folder	`pwd`, then `cd` to `dd8-lab/bopis-service`
`Missing script: "step"`	You are one folder up	`cd dd8-lab/bopis-service`
`Unsupported engine`, `SyntaxError`	Node is too old	`node --version` must say v20 or higher
`git: command not found`	git is not installed on this VM	Tell the facilitator: this lab needs it
`Please tell me who you are`	git has no name or email	`npm run setup` sets one; or `git config user.email "you@example.com"` and `git config user.name "Your Name"`
A check says "there is no starting commit"	`npm run setup` has not run	Run it (it refuses if the code or the documents were already edited: re-extract the zip)
A check says the files that are not yours were changed	Someone (or Claude) edited `scripts/`, `tests/existing`, the PRD or the Slack export	`git checkout -- <the file>`
A check says uncommitted changes	You have not committed the document or change	`git status --short`, then `git add -A` and `git commit -m "..."`
A check says the spec has been committed only once	The AI's draft and your review are in one commit	Commit the draft first, then commit the review. If they are mixed, `npm run checkpoint -- 2`, then redo the review
A check says the code does not do what the spec says	The code changed behaviour somewhere	Read the item and the answer it shows; find the line you changed that the spec does not ask for
`npm test` fails in `tests/existing`	The behaviour changed (these tests are the record of today's behaviour)	Find the change. Do not edit the test
`npm run trace` says the tool does not exist	You have not built it yet	That is milestone 7
The tests pass but a check says they do not notice a mistake	Your tests would stay green if the rule were wrong	Write a test that fails when the rule changes by one
`git stash list` shows something at the end	A stash was never popped	`git stash pop`
Claude edited files it should not (`scripts/`, `tests/existing`)	It ignored `CLAUDE.md`	`git checkout -- scripts tests/existing`, then restate the constraint

Part 6 · The kit, mapped

You get the kit on the day. This is what is in it, so that nothing is a surprise.

The folders

```
dd8-lab/
├── README.md                setup, in one page
├── bopis-service/           ← the ONLY folder you open in VS Code
│   ├── LAB-GUIDE.md         your guide: the nine milestone cards
│   ├── LAB-LOG.md           your evidence log
│   ├── CLAUDE.md            the project's context file (Claude Code reads it)
│   ├── package.json         the scripts (`npm run ...`); no dependencies
│   ├── docs/                prd.md and slack-thread.md (the sources), then your own documents
│   ├── spec/                your spec goes here
│   ├── src/bopis/           the shipped code: eligibility.js and catalog.js
│   ├── tests/
│   │   ├── baseline/        the lab's own sanity tests
│   │   ├── existing/        the team's tests of the shipped code
│   │   └── spec/            your tests, named after requirements
│   ├── tools/               your traceability check goes here (trace.mjs)
│   └── scripts/             the lab's runner (step, hint, progress, checkpoint, po)
├── checks/                  the hidden checks, and the answer key they compare with
├── checkpoints/             the reference solution after each milestone
└── hints/                   the three hints for each milestone, the reference briefs, the
product owner's answers
```

Why only `bopis-service`? Claude Code reads every file in the folder you open. If you opened `dd8-lab`, it would find the reference solutions, and it would use them. The folders next to `bopis-service` hold the answers on purpose.

The code you work on

`src/bopis/eligibility.js` has one function:

```
isBopisEligible(item, ctx)   // returns true or false
```

- `item` is `{ sku, name, category, weightKg, price, shelfLifeDays, hazmat, ageRestricted, supplier_recall }`. Only `sku` and `category` are always present, and a missing field counts as "no".
- `ctx` is `{ quantity = 1, storeStock = Infinity, storeOpen = true, ageVerified = false }`.
- It is **pure**: no clock, no network, no files. The same input always gives the same output.

`src/bopis/catalog.js` has the list of eleven categories and a few helpers. Not every helper is used by the eligibility rules. Whether a helper is *used* is a different question from whether it *exists*.

What you write, and what you leave alone

You write	You leave alone
`docs/drift-report.md`, `docs/spec-review.md`, `docs/tasks.md`, `docs/red-green.md`	`docs/prd.md`, `docs/slack-thread.md` (the sources)
`spec/bopis-eligibility.md`	`scripts/` (the lab's runner)
`src/bopis/eligibility.js` (and new files in `src/bopis/`)	`tests/baseline/`, `tests/existing/`
`tests/spec/*.test.js`	`package.json`
`tools/trace.mjs`, `LAB-LOG.md`	`checks/`, `checkpoints/`, `hints/` (not even open)

If a file you may not edit shows up as changed, put it back with `git checkout -- <file>`.

How the checks work

- **For documents** (milestones 1 to 4, and the notes in 3 and 8), the check reads your file and judges what it *says* against a hidden list of facts about the PRD, the Slack thread and the code. It does not judge your prose, and it never prints the answer.
- **For code** (milestone 5 and the homework), the check *runs* your function on about six thousand sample items and compares it with a hidden answer key.
- **For tests** (milestones 5, 6 and the homework), the check puts slightly wrong versions of the rules in place of your code and runs your tests against them. A test that stays green when a rule is wrong is not testing that rule.
- **For your traceability tool** (milestone 7), the check runs it on a dozen small projects built on the spot, each with one thing wrong.
- **For your history**, the check reads your git commits.
- **Nothing is sent anywhere.** There is no network, no API key and no account.

What is real and what stands in

	In this lab	In real life
The AI	**Claude Code, the real thing**	The same
The retailer, the people, the rules, the numbers	Made up	Your own
The Slack thread and the product owner	A file, and a command that prints the answers	Real people, whom you ask
The traceability check	A script **you** write	A script in your pipeline, run on every pull request
The requirements store	A Markdown file in the repository	A Markdown file, plus a tracker such as Azure Boards

Part 7 · Glossary

Term	Plain-English meaning
Acceptance criterion	One case a requirement must satisfy, written so that a test can be made from it. Usually Given / When / Then
BOPIS	Buy online, pick up in store
Changelog	A list inside the spec of every change to it, with the reason. It makes a change visible
Checkpoint	The reference solution after a milestone. `npm run checkpoint -- N` puts it in your folder, as git commits
Characterisation	Writing down what code actually does today, as tests, whether or not that is what it should do
Drift	The gap that opens when the code, the PRD and the team's decisions stop agreeing
Drift check	A check that flags a change to the code with no change to the spec
Given / When / Then	A way to write a case: a starting situation, an event, and the result you can observe
Hint ladder	Three hints and then a reveal for each milestone, taken one at a time
Non-goals	What the spec says it will not do. They stop anyone building more than was asked
Orphan (ID)	A requirement ID that appears in code or tests but is not in the spec
Owner	The named person who approves every change to the spec
PRD	Product requirements document: the prose that says what to build and why
Pulse	The one line `npm run progress` prints for pasting in the chat
Red / green	A test that fails (red) and then passes (green). A test should be seen red first
Reference brief	A good prompt for a milestone, shown at the check-ins
Requirement ID	A name such as `REQ-014` that never changes and is never reused. It travels with the requirement into tasks, code comments and test names
Source of truth	The one document everyone builds from and checks against
Spec	A structured document of requirements with IDs and criteria, precise enough to build and test from
Stash	A git shelf for uncommitted changes. `git stash` puts them there, `git stash pop` brings them back
Sync	Another word for using a checkpoint to catch up
Traceability	A link from each requirement to the code that implements it and the tests that verify it
Vague requirement	A sentence that cannot be tested as written ("fast", "most large items"). Flag it, do not guess

Part 8 · Answers, reading and the cheat sheet

Answers: the warm-up

Exercise 1 (git). `git stash` puts the uncommitted change aside, so the file goes back to the last commit; `git stash pop` brings it back. If you forgot the `pop`, `git stash list` shows what is waiting.

Exercise 2 (a sentence made precise). "Orders over 50 dollars ship free" is hiding two questions. *Over* means strictly more than 50, so exactly 50 is not free; the sentence also does not say which currency or whether the 50 includes tax. Two good criteria:

- **Given** an order total of 50.01, **when** the shipping cost is worked out, **then** shipping is free (0).
- **Given** an order total of exactly 50.00, **when** the shipping cost is worked out, **then** shipping costs 4.99.

A third one on 49.99 would be fine. Whatever you chose, the test you can write from each is the point. If you wrote "a big order", the sentence is not testable yet.

Exercise 3 (red, then green). With `>=` instead of `>`, the test about exactly 50 dollars goes red, because 50 would then ship free. That test is the one that guards the boundary. The test about 50.01 would still pass, which is why one test on each side matters.

Answers: the self-check

1. A PRD is prose for deciding what to build; a spec is structured (IDs, criteria) so that humans and tools can build and test from it, and it is versioned.
2. The ID is how everything points at the requirement (tasks, code comments, test names) and how its history is followed. Reusing one makes old links lie.
3. It cannot be tested: "quickly" has no number. Flag it as too vague, ask the owner what it means, then make it testable or drop it.
4. "Enough items" and "cheap" have no numbers, so you cannot choose the inputs or the expected result.
5. Because an unreviewed AI interpretation is a guess with the spec's authority. Once it is in the spec, the code, tests and people all build on it.
6. Drift. A drift check that flags a change to `src/` with no change to `spec/` would catch it.
7. It proves nothing: it may be checking something always true. Take the fix away and run it; or make the rule slightly wrong and see whether any test fails.
8. It names the requirement the test checks, so `--grep REQ-014` runs it, and a traceability script can find it.

Frequently asked questions

Is the spec just documentation? No. Documentation describes what exists. A spec says what is wanted, including what is not built yet, and code and tests are made from it and traced back to it.

Do I have to write the spec myself? No. Ask the AI to draft it, but make it flag what it cannot test, and review it with the owner. The draft is the easy part; the review is the work.

What if the product owner is not available? Then the vague sentence stays in the flagged list as an open question. That is better than a guess. In the lab the owner answers with `npm run po`.

What if the code is right and the PRD is wrong? Then the spec follows the decision, the PRD is superseded, and the changelog says why. Do not edit the code to match an old document without asking.

Why test names that start with an ID? So that a tool can find them without reading your mind, and so that `--grep` runs one requirement's tests.

Why does the lab compare my code with a hidden answer key? Because tests written by the same person who wrote the code share its blind spots. An independent key shows when your code does something the spec did not ask for.

What does the traceability check not tell me? That the code is *right*. It tells you each requirement has *some* code and *some* test. The test being able to fail is a separate check, and the review is a third.

Can I use my own editor or AI tool? The checks only look at files and git, so yes. Claude Code is the one the session is built around.

Reading list

- **Spec-driven development** in practice: GitHub's open-source *Spec Kit* describes a workflow of spec, plan, tasks and implementation. Skim its documentation for the shape of the idea.
- **Given / When / Then**: the structure comes from behaviour-driven development. Any introduction to Gherkin shows the format.
- **Traceability** and **requirements management**: the general idea predates AI by decades. A requirements-engineering primer is worth an hour.
- For tests that can fail: any introduction to **mutation testing** (making the code slightly wrong to see if tests notice). The lab does a small version of it for you.
- Earlier in the programme: the DD7 handbook's Part 3 on characterisation tests and on "a green test run only tells you about what someone tested".

Apply it at work: ten questions for any AI-assisted change

1. Which requirement is this change for, and where is its ID written?
2. Who owns the spec, and did they approve the change to it?
3. Is there a sentence in the PRD that the AI interpreted, and has the owner confirmed the interpretation?
4. Does each acceptance criterion have a test, and does the test name say which requirement it checks?
5. Was each new test red before the fix?
6. If I made the rule slightly wrong, would any test notice?
7. Does every requirement have code and a test, and does every ID in the code have a requirement?
8. Did the code move without the spec moving? If so, which is right?
9. Where did the last decision from a chat thread end up? Is it in the spec?
10. What would I check first if this change turned out to be wrong in production?

One-page cheat sheet

Where am I? `pwd` (should end in `bopis-service`) · `git status --short` · `git log --oneline`

I want to...	Command
Run all the tests	`npm test`
Run one requirement's tests	`npm test -- --grep REQ-003`
Run every requirement-linked test	`npm test -- --grep REQ-`
Check milestone N	`npm run step -- N`
Check everything, get the line for the chat	`npm run progress`
Get a hint (early, again, reference)	`npm run hint -- N` (`--now`, `--again`, `--reveal`)
Catch up for free	`npm run checkpoint -- N`
Ask the product owner	`npm run po`
Run my traceability check	`npm run trace` (`-- --json`, `-- --base <commit>`)
Save a document or change	`git add -A` then `git commit -m "..."`
See what I changed	`git diff`
Put changes aside / back	`git stash` / `git stash pop`
Put one file back as it was	`git checkout <commit> -- <file>`
Find the starting commit	`git log --oneline` (the bottom one), or `cat .lab/baseline-commit`
Make a bad commit disappear from the history	`git reset --soft <starting commit>`, then commit again

The shape of a requirement: `## REQ-001: Title` · one line · **Given** a situation, **When** an event, **Then** a result with a number in it.

Remember: a spec records what is wanted, including what is not built · the owner decides, the AI suggests, git remembers · carry the ID in the task, the code comment and the test name · write the test first and watch it fail · make the table a script.